

VPN, Made Simple

Every idea explained in plain words — and why it works that way

No jargon dumps. Each point is one simple sentence, followed by a green “💡 Why?” box that explains the reason behind it. By the end you’ll understand exactly how our app takes your phone traffic and makes it come out in another country — and why every piece exists.

Plain English

A “Why?” for everything

Diagrams

Interview prep

Our app: Shield VPN (Flutter) • **Our server:** a rented computer in Sweden running WireGuard

1. What is a VPN, really?

A VPN sends your internet traffic through **another computer** (a “server”) somewhere else, inside a **locked box** nobody can open on the way.

💡 **Why?** Two problems get solved at once. **(1) Privacy:** the locked box means your Wi-Fi, your internet company, or a hacker on the same network can’t see what you’re doing. **(2) Location:** because your traffic comes *out* of the server, websites think you are wherever the server is — not where you actually are.

The whole thing in one line: your phone → locked box → server in another country → the internet. Everything below is just *how* each arrow works and *why* it’s needed.

2. What we built, step by step

1 We rented a computer in Sweden (an AWS server).

💡 **Why?** A VPN needs a computer that lives somewhere else to be the “exit door” for your traffic. We can’t use your phone for that — it has to be a machine sitting in a data center that’s always on.

2 We opened one “door” on it: port 51820.

💡 **Why?** By default a server blocks all incoming connections for safety. Our VPN software listens on door number 51820, so we had to unlock exactly that one door — and no others — so the app can reach it while keeping the server secure.

3 We installed WireGuard and made two keys.

💡 **Why?** WireGuard is the software that builds the locked box. It needs a pair of keys (one for the server, one for your device) so each side can prove who it is and lock/unlock the box — like giving each side a matching lock.

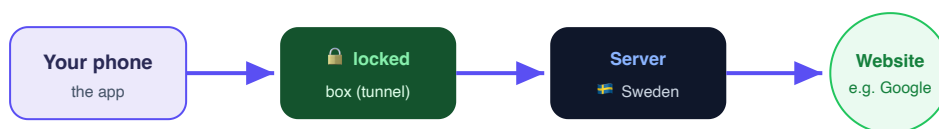
4 We tested it — our IP changed to Sweden.

💡 **Why?** Before writing any app, we proved the server works by connecting a laptop to it. Our public address (IP) became the Swedish server’s address. That confirmed the hardest part is done, so any later bug must be in the app, not the server.

5 We built the Flutter app around it.

💡 **Why?** The server does the real work, but users need a friendly button. Flutter lets us build one app for both iPhone and Android, with the Connect button, server list, and speed test.

3. How your data travels (with why at every step)



Your traffic goes into a locked box, travels to the server, gets unlocked there, then goes to the website. The reply comes back the same way.

Step A — the app grabs all your traffic first.

💡 **Why?** The phone is told “send everything through the VPN.” If even a little traffic escaped around the VPN, that part would be exposed — so it has to catch *all* of it to keep you fully private.

Step B — it locks each piece of data (encryption).

💡 **Why?** Data travels in small pieces called **packets**. Each one is scrambled so that anyone watching the network sees only gibberish. That’s the entire point of a VPN — without the lock, there’s no privacy.

Step C — it wraps the locked piece in a new envelope addressed to the server.

💡 **Why?** The outside of the envelope must show a real address so the internet can deliver it. So the only thing visible is “your phone → the VPN server.” The real destination (the website) is hidden *inside* the locked part, so no one on the way learns where you’re actually going.

Step D — the server unlocks it and sends it to the real website.

💡 **Why?** Only the server has the matching key, so only it can open the box. It then forwards your request to the website on your behalf — acting as your stand-in.

Step E — the website replies to the server, which locks it and sends it back to you.

💡 **Why?** The website only ever talked to the server, so it replies to the server. The server re-locks the answer and sends it back through the tunnel, and your phone unlocks it. The website never learns your real address.

4. The key pieces — each one, and why it exists

4.1 The “grabber” (TUN)

A special virtual network card that catches all your traffic before it leaves the phone.

💡 **Why?** Something has to intercept your data *before* it goes out normally, so the VPN can lock it. This virtual card is that interception point. Apps don't even notice — they send data the usual way, and it quietly gets rerouted into the VPN.

4.2 The lock and key (encryption + keys)

Each side has a **public key** (shareable) and a **private key** (secret).

💡 **Why?** You need a way to lock data so *only* the server can open it, and to be sure you're really talking to your server (not a fake one). The public key is like an open padlock you hand out; anyone can lock a box with it, but only your secret private key opens it. Swapping padlocks lets both sides send boxes only the other can open — and proves each side's identity.

4.3 The “door number” (port 51820)

The VPN listens on one specific numbered door on the server.

💡 **Why?** One computer runs many services (web, email, VPN...). Port numbers are how it knows which incoming traffic belongs to which service — like apartment numbers at one street address. The VPN uses 51820 so the server knows “this is VPN traffic.”

4.4 The stand-in address (NAT)

The server swaps your private tunnel address for its own public address on the way out.

💡 **Why?** Inside the tunnel you have a fake address (like `10.7.0.2`) that doesn't work on the open internet. So the server puts *its* real address on your traffic and writes down a note so it can send the reply back to the right person. This is also *why* the website sees the server instead of you.

🏠 **Think of a hotel front desk:** you (room 10.7.0.2) order food. The desk calls the restaurant using the *hotel's* number, and when the food arrives it checks its notebook and brings it to your room. The restaurant only knows the hotel.

4.5 The address book (DNS)

Before visiting a site, your phone looks up its number, and we send that lookup *through* the tunnel too.

💡 **Why?** Website names like `google.com` must be turned into numbers (IP addresses) first. If that lookup went outside the locked box, your internet company would still see the *names* of every site you visit — so we route it inside the tunnel to keep even that private. (Leaking it is called a “DNS leak.”)

5. Why fast vs. sneaky: UDP, and the three protocols

Why WireGuard sends data as “UDP”

UDP is the “just send it, don’t wait” style of delivery.

💡 **Why?** A VPN is carrying other traffic inside it that *already* handles lost pieces. If the VPN also waited and double-checked every piece, you’d get two systems re-sending at once and everything would crawl. UDP stays out of the way, so it’s fast and light on battery — great for phones.

The three ways to build the tunnel — and why you’d pick each

Option	What it is	Why choose it
WireGuard	Modern, tiny, very fast	Best default — fastest, best battery, simplest. Use it unless something blocks it.
OpenVPN	Older; can disguise itself as normal website traffic (HTTPS)	Use as a backup when a network blocks WireGuard, because it’s much harder to tell apart from everyday browsing.
Custom / obfuscated	WireGuard/OpenVPN wrapped in a disguise	Use in places with heavy blocking — it makes the tunnel look like ordinary traffic so it slips through.

💡 **Why? Why not just always use one?** Different networks fight back differently. A normal Wi-Fi is happy with fast WireGuard. A strict office or country firewall may block it — so you fall back to something that looks innocent. You keep the same strong lock; you only change the *disguise*.

💡 **Why? Why never invent your own encryption for “custom”?** Making a secure lock is incredibly hard and a tiny mistake is invisible until someone breaks in. So “custom” always means a new *disguise* around a lock experts have already tested for years — never a home-made lock.

6. What if we add more servers?

One server means one country and a limited amount of traffic. Real apps have many, which means we need a “**dispatcher**” (a backend).

💡 **Why?** With one server, the app can just hold its details. With hundreds, the app can’t know them all, which one is closest, or which is busy. A backend keeps the full list and hands each user the best one — like a taxi dispatcher assigning the nearest free car.

What the dispatcher (backend) must do — and why

- **Keep a list of all servers** — so the app can show the country menu and know what’s available.
- **Give out a fresh config when you connect** — so each user gets their own keys on the chosen server, and we can cut them off when they stop paying.

- **Check your subscription first** — so only paying users get access (this can't be done safely inside the app alone — it must be checked on the server).
- **Pick the best server (near + not busy)** — so you get the fastest connection instead of a random one.
- **Automate adding new servers** — so launching a new country takes one command, not a day of manual setup.
- **Watch bandwidth cost** — because moving lots of data is the real bill; AWS is too pricey at scale, so production uses cheaper unlimited-traffic hosts.

7. Interview questions (simple answers)

In one sentence, what does a VPN do?

It sends your traffic through another computer inside an unbreakable locked box, so no one nearby can read it and websites see that other computer instead of you.

Why does the website see the server's location, not yours?

Because the server forwards your request using *its own* address (that's NAT). The website only ever talks to the server, so it only knows the server.

Why can't the Flutter app do the VPN by itself?

Phones only let a special system component grab and reroute all traffic. The app can't touch that directly, so the real tunnel runs in native code and the Flutter app just controls it with a button.

Why UDP and not the reliable TCP?

The traffic inside the tunnel already fixes lost pieces. If the tunnel also did that, two systems would re-send at once and slow everything down. UDP is fast and gets out of the way.

What is a DNS leak and why does it matter?

It's when the "look up the website's number" step happens outside the tunnel. Even though the data is locked, the leak reveals the *names* of sites you visit — so we route that lookup through the tunnel too.

Why do we need a backend once we have many servers?

The app can't track hundreds of servers or decide which is best. The backend stores them all, checks your subscription, and hands you the nearest free one with a fresh, temporary key.

Why never write your own encryption?

Secure encryption is extremely hard and mistakes are invisible until someone exploits them. We reuse locks that experts have tested for years and only change how the tunnel is *disguised*.